06f — API Security Best Practices

Key Concepts

- **Authentication** — verifying identity ("who are you?")
- **Authorization** — verifying permission ("what are you allowed to do?")
- **Rate Limiting** — capping requests per user/IP/endpoint
- **Input Validation** — checking type, length, format at the API boundary
- **DTO** — Data Transfer Object; controls exactly what fields get serialized in responses
- **HSTS** — HTTP Strict Transport Security; tells browsers to never use HTTP for your domain
- **Secure Headers** — HTTP response headers that harden the browser/client layer
- **429 Too Many Requests** — correct status when rate limit is exceeded
- **Information Disclosure** — leaking internal details (stack traces, DB schema) in error responses

API Security Checklist

Practice	Why
Auth on every endpoint	Prevents unauthorized access
Rate limiting per user + endpoint	Prevents brute force and DoS
Input validation at boundary	Prevents injection, bad data reaching DB
HTTPS only + HSTS	Prevents interception
Minimal response payload (DTOs)	Prevents data leakage
Secure headers	Hardens HTTP layer
Generic errors externally, full logs internally	Prevents information disclosure
API versioning	Prevents breaking clients on updates
Logging & monitoring (alert on 401/500 spikes)	Detect and respond to attacks

How It Works

Auth Flow (every request)

1. Request arrives at endpoint
2. Check: is token valid? (Authentication)
3. Check: does this user own/have permission for this resource? (Authorization)
4. Fail closed — if check errors, deny by default

Rate Limiting

```
User exceeds limit → 429 Too Many Requests
Response header: Retry-After: 60
```

Input Validation (at controller level)

- Reject before business logic touches it
- Check: type, length, format, range
- Example: age must be integer 0–120, email must match regex

Minimal Response (DTO pattern)

```
Entity has 20 fields → DTO exposes 3 fields → client gets only 3
Never expose: password hashes, internal IDs, scoring fields, third-party keys
```

Error Handling

```
External response: { "error": "Unexpected error. Ref: ERR-4821" }
Internal log: full stack trace, user ID, request details
```

Secure Headers

Header	Purpose
`Strict-Transport-Security`	Forces HTTPS
`X-Content-Type-Options: nosniff`	Prevents MIME sniffing
`X-Frame-Options: DENY`	Prevents clickjacking
`Content-Security-Policy`	Restricts resource loading

In .NET: use **NWebSec** library to add these automatically.

HTTP Status Codes

Situation	Code
Not logged in	401 Unauthorized
Logged in, no permission	403 Forbidden
Resource not found (or hidden)	404 Not Found
Validation failed	400 Bad Request
Server error	500 Internal Server Error

Situation	Code
Rate limit exceeded	429 Too Many Requests

Industry Examples

- **Twitter API** — 900 requests per 15 min per endpoint; 429 on excess
- **GitHub** — private repos return 404, not 403 (prevents enumeration)
- **Average breach detection time** — 207 days without proper monitoring (OWASP A09)

Interview Questions

Q: Walk me through security measures before shipping a REST API. A: Input validation, rate limiting, auth/authz on every endpoint, HTTPS + HSTS, minimal DTOs, secure headers, generic errors externally with full internal logs, monitoring with alerts on 401/500 spikes.

Q: Why use DTOs instead of returning entity objects directly? A: Entities often contain internal fields (password hashes, scoring, third-party keys). DTOs explicitly define what the client sees — no accidental data leakage.

Q: Why should errors return generic messages externally? A: Stack traces reveal file paths, DB schema, framework versions — a roadmap for attackers. Log internally, return a reference ID externally.

Q: What's the difference between authentication and authorization? A: Authentication = who are you (identity). Authorization = what are you allowed to do (permission). Both must be checked on every endpoint.

Q: Why return 404 instead of 403 for unauthorized resource access? A: 403 confirms the resource exists. 404 reveals nothing — prevents enumeration attacks.